We've picked a concrete representation for `Location` here that includes the full input, an offset into this input, and the line and column numbers, which can be computed lazily from the full input and offset. We can now say more precisely what we expect from `label`. In the event of failure with `Left(e)`, `errorMessage(e)` will equal the message set by `label`. This can be specified with a `Prop`:

```
def labelLaw[A](p: Parser[A], inputs: SGen[String]): Prop =
  forAll(inputs ** Gen.string) { case (input, msg) =>
    run(label(msg)(p))(input) match {
      case Left(e) => errorMessage(e) == msg
      case _ => true
    }
  }
```

What about the `Location`? We'd like for this to be filled in by the `Parsers` implementation with the location where the error occurred. This notion is still a bit fuzzy—if we have `a or b` and both parsers fail on the input, which location is reported, and which label(s)? We'll discuss this in the next section.

9.5.2 Error nesting

Is the `label` combinator sufficient for all our error-reporting needs? Not quite. Let's look at an example:

```
val p = label("first magic word")("abra") **
        " ".many **                                  <-- Skip whitespace
        label("second magic word")("cadabra")
```

What sort of `ParseError` would we like to get back from `run(p)("abra cAdabra")`? (Note the capital `A` in `cAdabra`.) The immediate cause is that capital `'A'` instead of the expected lowercase `'a'`. That error will have a location, and it might be nice to report it somehow. But reporting *only* that low-level error wouldn't be very informative, especially if this were part of a large grammar and we were running the parser on a larger input. We have some more context that would be useful to know—the immediate error occurred in the `Parser` labeled `"second magic word"`. This is certainly helpful information. Ideally, the error message should tell us that while parsing `"second magic word"`, there was an unexpected capital `'A'`. That pinpoints the error and gives us the context needed to understand it. Perhaps the top-level parser (`p` in this case) might be able to provide an even higher-level description of what the parser was doing when it failed (`"parsing magic spell"`, say), which could also be informative.

So it seems wrong to assume that one level of error reporting will always be sufficient. Let's therefore provide a way to *nest* labels:

```
def scope[A](msg: String)(p: Parser[A]): Parser[A]
```

Unlike `label`, `scope` doesn't throw away the label(s) attached to `p`—it merely adds additional information in the event that `p` fails. Let's specify what this means exactly. First, we modify the functions that pull information out of a `ParseError`.